

Design for interface “PaymentGateway”

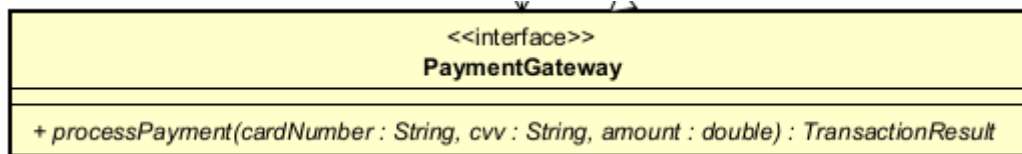


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	N/A	N/A	N/A	N/A

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	processPayment	TransactionResult	A standardized interface operation to process credit card payments.

Parameter:

- cardNumber: No default value, the customer's credit card number.
- cvv: No default value, the 3-digit security code of the card.
- amount: No default value, the total order amount to be charged.

Exception:

- InvalidCardException if the card format is invalid before initiating the request.
- PaymentTimeoutException if the payment gateway does not respond within the acceptable time limit.

Method:

- As an interface, this only defines the contract. Subsystem-specific classes (e.g., PayPalGatewayService) will provide the implementation.

Design for class “PaypalGatewayService ”

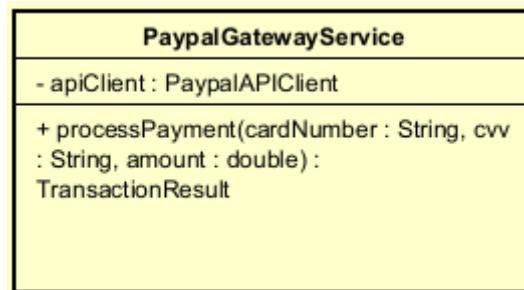


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	apiClient	PaypalAPIClient	null	The client object used to establish connections and send HTTP requests to the external PayPal server.

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	processPayment	TransactionResult	Implements the PaymentGateway interface to handle PayPal-specific payment logic.

Parameter:

- cardNumber: No default value, the credit card number.
- cvv: No default value, the CVV code.
- amount: No default value, the amount to be charged.

Exception:

- `ExternalServerException` if the external PayPal API returns a 500 Internal Server Error or refuses the connection.

Method:

1. Initialize a `PaypalRequestDTO` object using the input parameters (`cardNumber`, `cvv`, `amount`).
2. Invoke `apiClient.sendTransactionRequest(requestDTO)`.
3. Receive the returned `PaypalResponseDTO` object.
4. Evaluate the `status` attribute of the response. If `status` is "SUCCESS", set `isSuccess = true`. Otherwise, extract the `errorCode` to populate the error message.
5. Instantiate and return a `TransactionResult` object.

Design for class“PaypalAPIClient ”

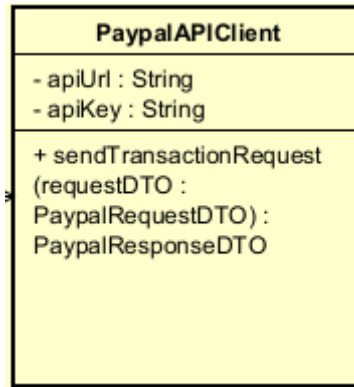


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	apiUrl	String	"https://api.paypal.com/v1/payments"	The endpoint URL of the external PayPal REST API.
2	apiKey	String	null	The authentication token/key required to access the PayPal API.

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	sendTransactionRequest	PaypalResponse DTO	Handles the network communication, sending data to and receiving JSON from PayPal.

Parameter:

- requestDTO: No default value, the data transfer object containing the encrypted payload to be sent.

Exception:

- `ConnectionException` if the network is down or unreachable.

- `UnauthorizedException` if the `apiKey` is invalid or expired.

Method:

1. Serialize the `requestDTO` object into a JSON payload.
2. Construct an HTTP POST request targeting the `apiUrl` with the `apiKey` in the headers.
3. Execute the request and wait for the external server's response.
4. Parse the returned JSON string and pass it to the constructor of `PaypalResponseDTO`.
5. Return the newly created `PaypalResponseDTO`.

Design for class “PaypalRequestDTO”

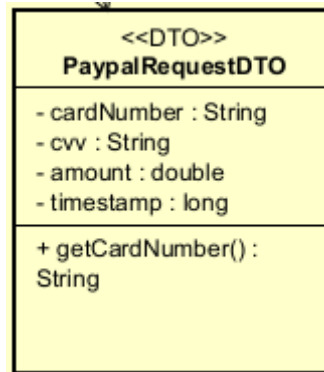


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	cardNumber	String	null	Customer's credit card number.
2	cvv	String	null	Security code.
3	amount	double	0.0	Transaction amount.
4	timestamp	long	Current System Time	The exact Unix timestamp when the request was generated.

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	PaypalRequestDTO	void	Constructor to encapsulate the data before transmission.
2	getCardNumber	String	Retrieves the card number.

Parameter:

- Constructor parameters map directly to the attributes listed above.

Exception:

- None.

Method:

- Constructor initializes the attributes. Getter methods return the encapsulated values.

Design for class “PaypalResponseDTO ”

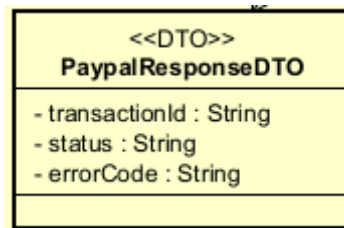


Table 1. Attribute design

#	Name	Data type	Default value	Description
1	transactionId	String	null	The unique identifier generated by the PayPal server.
2	status	String	null	The raw status string (e.g., "SUCCESS", "DECLINED").
3	errorCode	String	""	The specific error code provided by PayPal if the transaction fails.

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	PaypalResponseDTO	void	Constructor that parses the raw JSON into object attributes.

Parameter:

- jsonResponse: No default value, the raw JSON string returned from the HTTP request.

Exception:

- `JsonParseException` if the response format is invalid or corrupted.

Method:

- The constructor extracts `transactionId`, `status`, and `errorCode` from the provided `jsonResponse` string and assigns them to the class attributes.

Design for class “TransactionResult”

TransactionResult
- isSuccess : boolean - transactionId : String - errorMessage : String
+ TransactionResult(isSuccess : boolean, transactionId : String, errorMessage : String) : void + isSuccess() : boolean + getTransactionId() : String + getErrorMessage() : String

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	isSuccess	boolean	false	A flag indicating whether the payment was processed successfully.
2	transactionId	String	null	The standardized transaction ID for database storage and reconciliation.
3	errorMessage	String	""	Contains the failure reason or error message if the transaction fails.

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	TransactionResult	void	Constructor to initialize the final standardized result object.
2	isSuccess	boolean	Returns the success flag.
3	getTransactionId	String	Returns the transaction ID.
4	getErrorMessage	String	Returns the error message.

Parameter:

- isSuccess: The success status.
- transactionId: The transaction ID.

- `errorMessage`: The error details.

Exception:

- None.

Method:

- Constructor assigns the input parameters directly to the class attributes. Getters return the attribute values to the caller.